МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

« Базовые SQL-запросы и фильтрация данных »

Выполнил: студент группы БВТ2504
Воронин А.А.

Москва, 2026 г.

1. Постановка задачи

Целью лабораторной работы является освоение многотабличных запросов к базе данных маркетплейса в СУБД PostgreSQL: соединения таблиц с помощью JOIN, внешних соединений для поиска строк без соответствий, а также группировки данных с агрегатными функциями и фильтрации групп секцией HAVING.

В рамках работы необходимо выполнить следующие задания технического задания: написать запрос с внутренним соединением не менее трёх таблиц (товар — категория — продавец); написать запрос с LEFT JOIN, показывающий строки без соответствий в связанной таблице (товары без отзывов); применить HAVING для фильтрации групп (продавцы с выручкой выше заданной).

Все запросы выполняются к базе данных marketplace, содержащей схему маркетплейса из лабораторной работы №1 и тестовые данные, добавленные в лабораторной работе №2.

2. SQL Зпрос.

2.1. Внутреннее соединение трёх таблиц

вывести товары вместе с названием категории и именем продавца. Данные о товаре, категории и продавце хранятся в трёх разных таблицах, поэтому для получения осмысленного результата их необходимо соединить.

```sql
-- 1
SELECT p.title, c.name AS category, s.store_name
FROM products p
JOIN categories c ON p.category_id = c.id
JOIN sellers s ON p.seller_id = s.id;
```

	title text	category text	store_name text
1	товар №1	манга	store_19
2	товар №2	посуда	store_1
3	товар №3	пк	store_2
4	товар №4	мебель	store_1
5	товар №5	плавательн…	store_16
6	товар №6	пк	store_19
7	товар №7	пк	store_15
8	товар №8	наушники	store_3
9	товар №9	пк	store_7
10	товар №10	посуда	store_15
11	товар №11	манхва	store_13
12	товар №12	наушники	store_13
13	товар №13	пк	store_6
14	товар №14	мебель	store_13
15	товар №15	спортивная	store_5

Пояснение. JOIN без уточнения означает INNER JOIN: в результат попадают только строки, для которых выполняется условие ON. Товар соединяется с категорией по c.id = p.category_id и с продавцом по s.id = p.seller_id. Товар без категории или продавца в выборку не попадёт. Псевдонимы таблиц (p, c, s) устраняют неоднозначность.

2.2. LEFT JOIN: товары без отзывов

Назначение: найти товары, на которые ещё не оставлено ни одного отзыва.

```sql
-- 2
SELECT p.title
FROM products p
LEFT JOIN reviews r ON p.id = r.product_id
WHERE r.id IS NULL;
```

	title text
1	товар №10…
2	товар №251
3	товар №264
4	товар №10…
5	товар №455
6	товар №11…
7	товар №209
8	товар №276
9	товар №606
10	товар №850

Пояснение. LEFT JOIN сохраняет все товары, а если отзывов нет, столбцы reviews заполняются NULL. Условие r.id is null оставляет только такие строки. Сравнение r.id = null не сработает, так как даёт UNKNOWN. При INNER JOIN такие товары были бы отброшены.

2.3. Группировка и агрегация по категориям

Назначение: для каждой категории получить количество товаров и среднюю цену.

```
-- 3
SELECT
    c.name,
    COUNT(p.id) AS products_count,
    AVG(p.price) AS avg_price
FROM categories c
LEFT JOIN products p ON c.id = p.category_id
GROUP BY c.id, c.name;
```

	name text	products_count bigint	avg_price numeric
1	спортивная	142	5330.4895774647887324
2	телефоны	63	5309.1584126984126984
3	одежда	0	[null]
4	плавательная	148	5154.8275000000000000
5	пк	153	5721.0332679738562092
6	манхва	129	5493.6086821705426357
7	дом	0	[null]
8	велосипеды	63	5592.6101587301587302
9	бутылки для во...	0	[null]
10	посуда	139	5353.6483453237410072
11	электроника	0	[null]
12	мебель	109	5523.3312844036697248
13	наушники	133	5702.5496240601503759
14	книги	0	[null]
15	манга	121	5570.6863636363636364
16	спорт	0	[null]

Пояснение. GROUP BY делит товары на группы по категориям, а агрегатные функции COUNT() и AVG() считают значения для каждой группы. В списке выборки допустимы только столбцы из GROUP BY и агрегатные функции. Использование LEFT JOIN позволяет вывести абсолютно все категории, даже те, в которых пока нет ни одного товара (для них COUNT вернет 0, а AVG — NULL).

2.4. Фильтрация групп с помощью HAVING

Назначение: вывести продавцов, суммарная выручка которых превышает заданный порог.(> 10000)

```sql
-- 4
SELECT
    s.store_name,
    SUM(oi.quantity * oi.unit_price) AS revenue
FROM sellers s
JOIN products p ON p.seller_id = s.id
JOIN order_items oi ON oi.product_id = p.id
GROUP BY s.id, s.store_name
HAVING SUM(oi.quantity * oi.unit_price) > 10000;
```

	store_name text	revenue numeric
1	store_6	24720165.00
2	store_15	16092765.00
3	store_17	15844950.00

Пояснение. ROUP BY делит строки по продавцам, а SUM(quantity * unit_price) считает выручку каждого. Условие по выручке нельзя поместить в WHERE: он выполняется до группировки, когда выручки ещё нет. HAVING работает после группировки и фильтрует готовые группы по результату агрегации.

2.5. Подзапросы трёх видов

Скалярный подзапрос

Назначение: найти товары дороже средней цены по всей базе.

```sql
-- 5.1
SELECT title, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

	title text	price numeric (12,2)
1	товар №1	9507.12
2	товар №3	8723.87
3	товар №5	6857.98
4	товар №6	9681.64
5	товар №7	7607.63
6	товар №9	6154.61
7	товар №10	8515.01
8	товар №13	8701.53
9	товар №14	8662.03
10	товар №16	9265.56
11	товар №19	7857.75
12	товар №20	8207.82
13	товар №24	5954.50
14	товар №25	5732.61
15	товар №26	7330.06

Пояснение. Подзапрос (select avg(price) from products) возвращает ровно

одно значение, которое сравнивается с ценой каждого товара. Он некоррелированный: вычисляется один раз и не зависит от внешнего запроса.

Подзапрос с IN

Назначение: вывести товары, принадлежащие продавцам с выручкой выше заданной (или товары из определённых категорий).

```sql
-- 5.2
SELECT title
FROM products
WHERE category_id IN (
    SELECT id FROM categories WHERE name = 'пк'
);
```

	title text
1	товар №3
2	товар №6
3	товар №7
4	товар №9
5	товар №13
6	товар №17
7	товар №28
8	товар №37
9	товар №44
10	товар №50
11	товар №57
12	товар №93
13	товар №95
14	товар №102
15	товар №111

Пояснение. Подзапрос возвращает набор значений, а предикат IN проверяет, входит ли в него значение из внешнего запроса. Это удобная замена цепочке условий через OR, когда список значений заранее неизвестен и определяется данными.

Подзапрос с EXISTS

Назначение: вывести продавцов, у которых есть хотя бы один товар.

```sql
-- 5.3
SELECT store_name
FROM sellers s
WHERE EXISTS (
    SELECT 1 FROM products p WHERE p.seller_id = s.id
);
```

	store_name text
1	store_1
2	store_2
3	store_3
4	store_4
5	store_5
6	store_6
7	store_7
8	store_8
9	store_9
10	store_10
11	store_11
12	store_12
13	store_13
14	store_14
15	store_15

Пояснение. Подзапрос коррелированный: он ссылается на s.id внешнего запроса и логически вычисляется для каждого продавца. EXISTS истинен, если подзапрос вернул хотя бы одну строку, при этом сами значения не важны, поэтому пишут SELECT 1.

2.6. Общие табличные выражения (CTE)

CTE для разбиения запроса на шаги

Назначение: рассчитать количество товаров у каждого продавца.

```sql
-- 6.1
WITH stats AS (
    SELECT seller_id, COUNT(*) AS total
    FROM products
    GROUP BY seller_id
)
SELECT s.store_name, stats.total
FROM stats
JOIN sellers s ON s.id = stats.seller_id;
```

	store_name text	total bigint
1	store_1	28
2	store_2	52
3	store_3	65
4	store_4	60
5	store_5	53
6	store_6	64
7	store_7	69
8	store_8	60
9	store_9	63
10	store_10	65
11	store_11	52
12	store_12	64
13	store_13	61
14	store_14	63
15	store_15	75

Пояснение. CTE, вводимое словом WITH, именует промежуточный результат и позволяет обращаться к нему как к обычной таблице. Запрос делится на логические шаги: сначала агрегация в CTE, затем соединение с таблицей продавцов.

Рекурсивный CTE

Назначение: Получить дерево категорий (от корневых к дочерним)..

```sql
-- 6.2
WITH RECURSIVE tree AS (
    SELECT id, name, parent_id
    FROM categories
    WHERE parent_id IS NULL

    UNION ALL

    SELECT c.id, c.name, c.parent_id
    FROM categories c
    JOIN tree t ON c.parent_id = t.id
)
SELECT * FROM tree;
```

	id bigint	name text	parent_id bigint
1	1	электроника	[null]
2	2	книги	[null]
3	3	одежда	[null]
4	4	дом	[null]
5	5	спорт	[null]
6	6	телефоны	1
7	7	пк	1
8	8	наушники	1
9	9	манга	2
10	10	манхва	2
11	11	плавательная	3
12	12	спортивная	3
13	13	мебель	4
14	14	посуда	4
15	15	велосипеды	5
16	16	бутылки для во...	5

Пояснение.

Рекурсивный CTE (WITH RECURSIVE) состоит из двух частей, объединённых UNION ALL. Первая (якорная) выбирает корневые категории (parent_id IS NULL). Вторая ссылается на сам CTE и на каждом шаге присоединяет дочерние категории. Рекурсия останавливается, когда новые строки перестают находиться.

2.7. Представления (VIEW)

Представление « Статистика по продавцам »

Назначение:

сохранить в базе готовый отчет по количеству товаров у продавцов.

```sql
-- 7.1
DROP VIEW IF EXISTS seller_stats;
CREATE OR REPLACE VIEW seller_stats AS
SELECT
    s.store_name,
    COUNT(p.id) AS products_count
FROM sellers s
LEFT JOIN products p ON p.seller_id = s.id
GROUP BY s.id, s.store_name;

SELECT * FROM seller_stats;
```

```
CREATE VIEW

Query returned successfully in 51 msec.
```

	store_name text	products_count bigint
1	store_18	67
2	store_6	64
3	store_13	61
4	store_8	60
5	store_16	65
6	store_1	28
7	store_12	64
8	store_14	63
9	store_3	65
10	store_20	24
11	store_2	52
12	store_9	63
13	store_4	60
14	store_7	69
15	store_15	75

Пояснение.

VIEW хранит не сами данные, а текст запроса. При каждом обращении выполняется SELECT, лежащий в его основе, поэтому результат всегда актуален. Команда DROP VIEW IF EXISTS используется для того, чтобы можно было безопасно перезапускать скрипт.

Представление «Статистика по категориям»

Назначение: сохранить сводку по категориям: количество товаров и средняя цена.

```sql
-- 7.2
DROP VIEW IF EXISTS category_stats;
CREATE OR REPLACE VIEW category_stats AS
SELECT
    c.name,
    COUNT(p.id) AS products_count,
    AVG(p.price) AS avg_price
FROM categories c
LEFT JOIN products p ON p.category_id = c.id
GROUP BY c.id, c.name;

SELECT * FROM category_stats;
```

```
CREATE VIEW

Query returned successfully in 49 msec.
```

	name text	products_count bigint	avg_price numeric
1	спортивная	142	5330.4895774647887324
2	телефоны	63	5309.1584126984126984
3	одежда	0	[null]
4	плавательная	148	5154.8275000000000000
5	пк	153	5721.0332679738562092
6	манхва	129	5493.6086821705426357
7	дом	0	[null]
8	велосипеды	63	5592.6101587301587302
9	бутылки для во...	0	[null]
10	посуда	139	5353.6483453237410072
11	электроника	0	[null]
12	мебель	109	5523.3312844036697248
13	наушники	133	5702.5496240601503759
14	книги	0	[null]
15	манга	121	5570.6863636363636364
16	спорт	0	[null]

Пояснение. Представление оформляет типовой аналитический запрос в готовую витрину. К нему можно обращаться как к обычной таблице: фильтровать, сортировать и соединять с другими таблицами, не прописывая каждый раз сложную агрегацию.

3. Выводы

В ходе лабораторной работы были разработаны и проверены многотабличные запросы к базе данных маркетплейса. С помощью внутреннего соединения трёх таблиц products, categories и sellers получен список товаров с названиями категорий и именами продавцов.

Запрос с LEFT JOIN и проверкой IS NULL позволил выявить товары без отзывов, то есть строки, не имеющие соответствия в связанной таблице.

Запрос с группировкой GROUP BY, агрегатной функцией SUM и секцией HAVING позволил отобрать продавцов, чья суммарная выручка превышает заданный порог.

В результате закреплено различие между INNER JOIN и LEFT JOIN, а также между WHERE и HAVING.